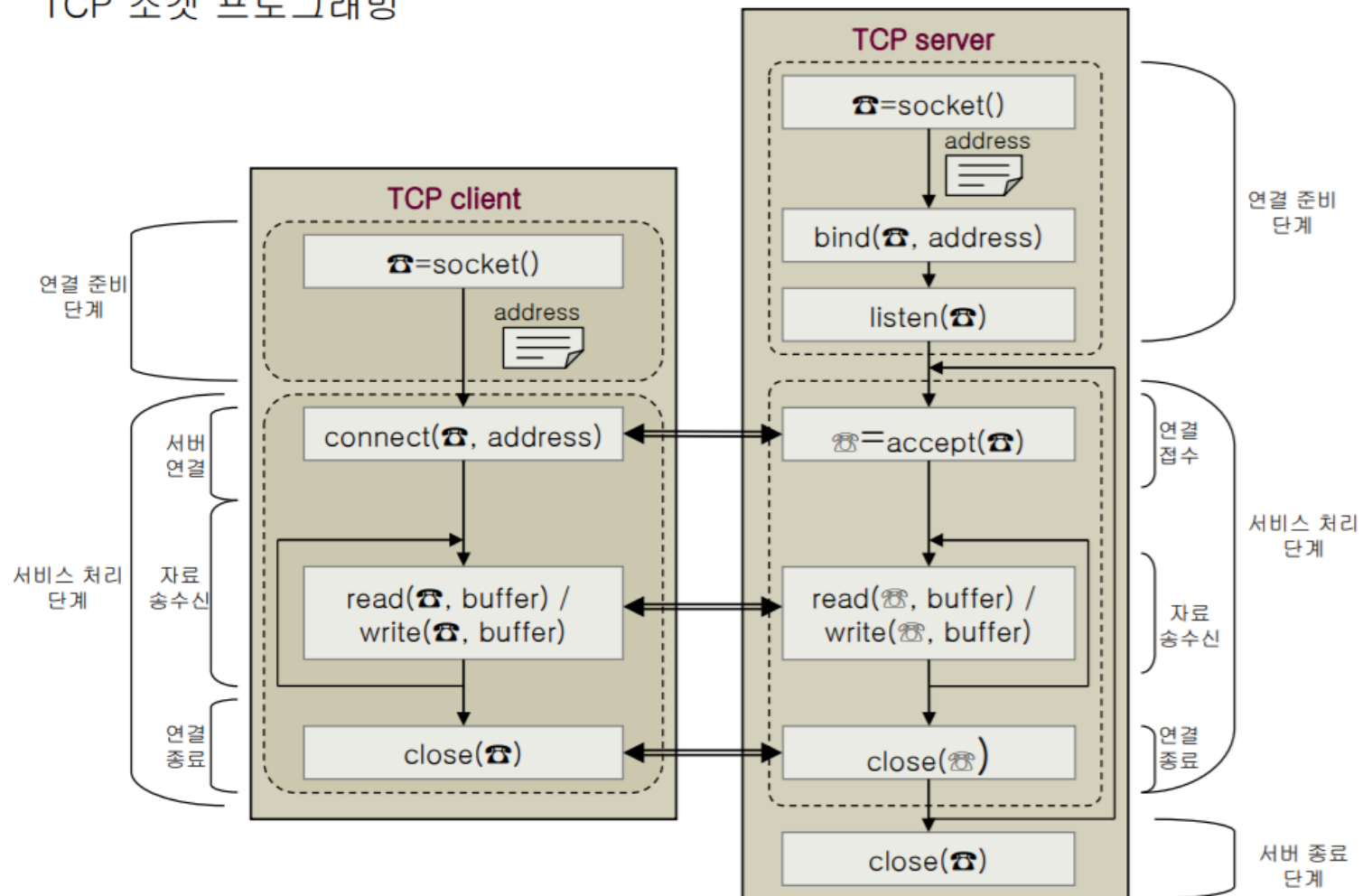


소켓 프로그래밍

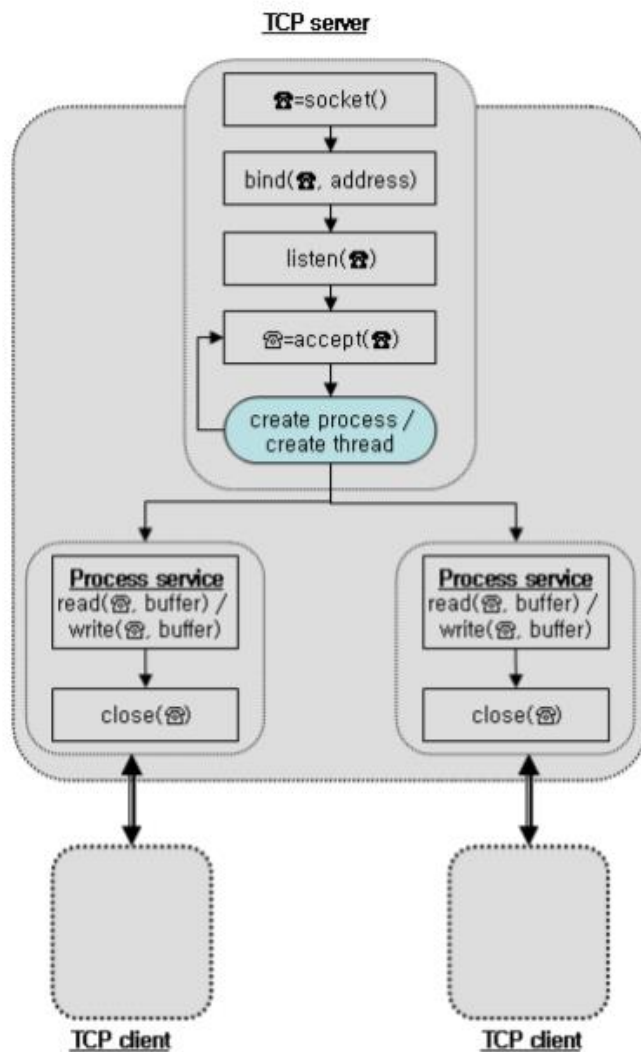
8장 멀티스레딩 방식 다중 서버

소켓 프로그래밍

TCP 소켓 프로그래밍



소켓 프로그래밍



```
main() {
    ....
    listen_s = socket(...);
    bind(listen_s, ...);
    listen(listen_s, ...);

    while(1) {
        // 클라이언트 요청 접수
        conn_s = accept(listen_s, ...);
        // 접수된 클라이언트 서비스를
        // 위한 쓰레드 생성
        pthread_create(..., do_service,...);
    }
}

// 쓰레드 함수
do_service(int conn_s) {
    // process service with client
    ....
}
```

소켓 프로그래밍

```
#include <stdio.h>
```

```
int sum = 0;
```

```
int
```

```
do_sum(int a)
```

```
{
```

```
    int    i;
```

```
    sum = 0;
```

```
    for (i = 0; i < a; i++) {
```

```
        sum = sum + i;
```

```
    }
```

```
    return (sum);
```

```
}
```

```
int main()
```

```
{
```

```
    int    res;
```

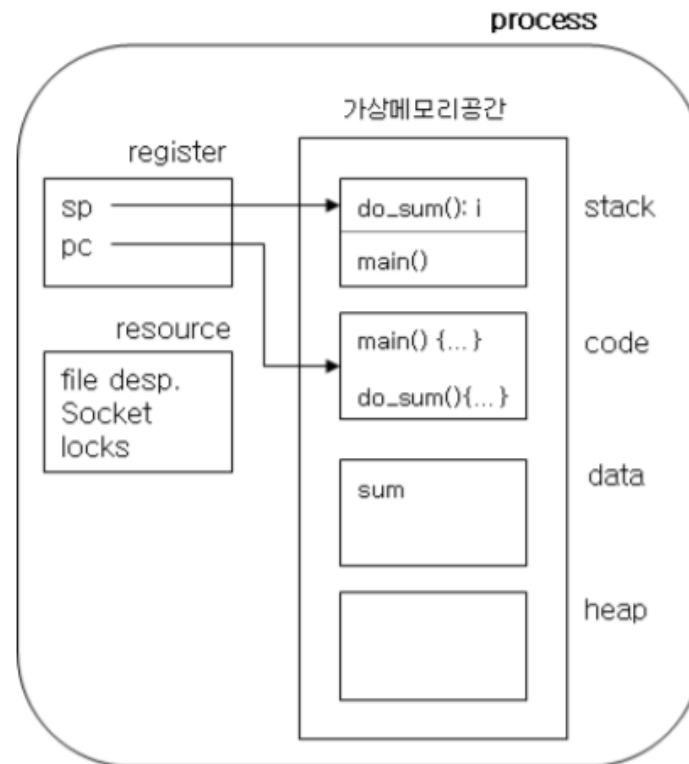
```
    int    a = 10;
```

```
    res = do_sum(a);
```

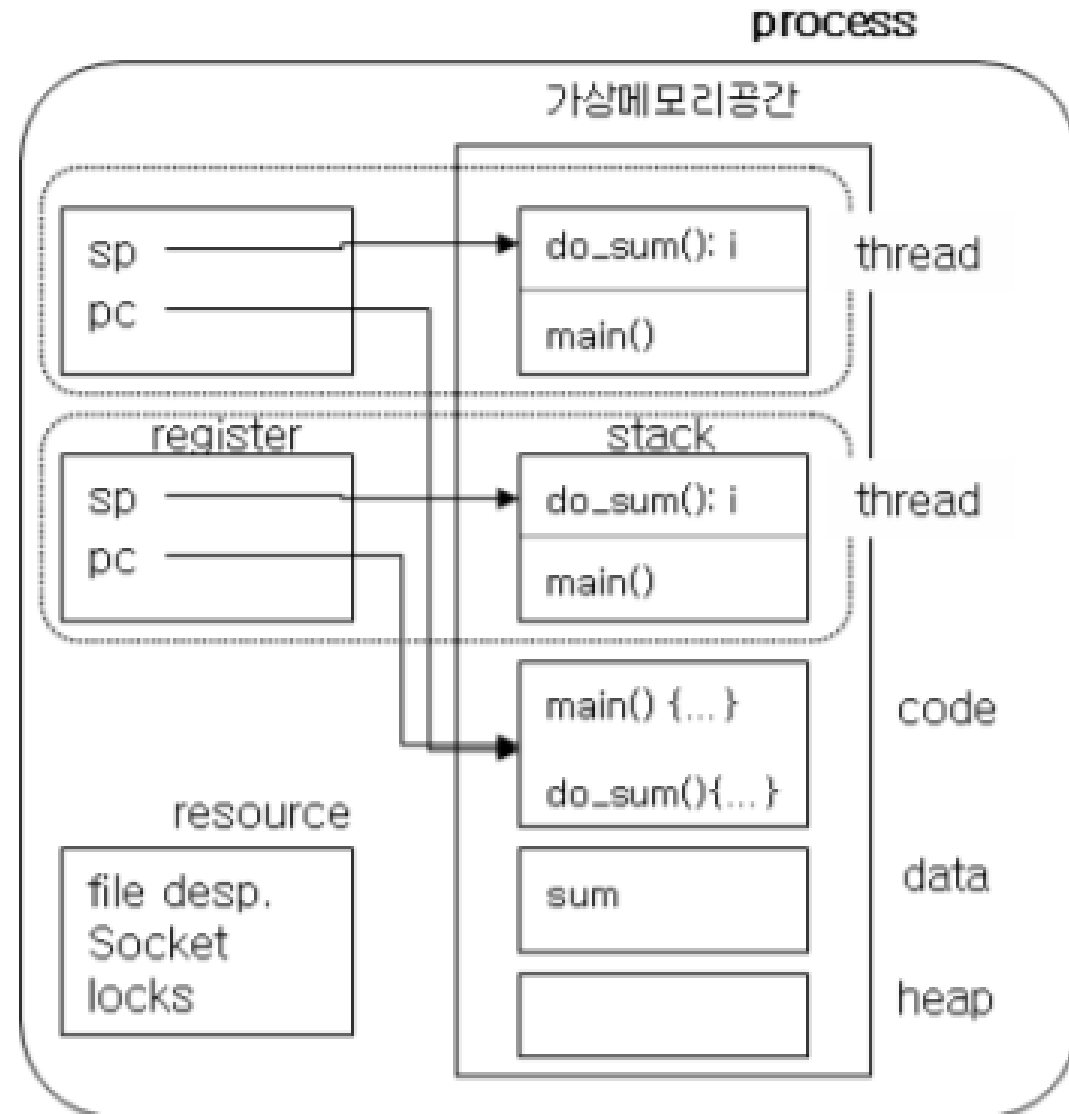
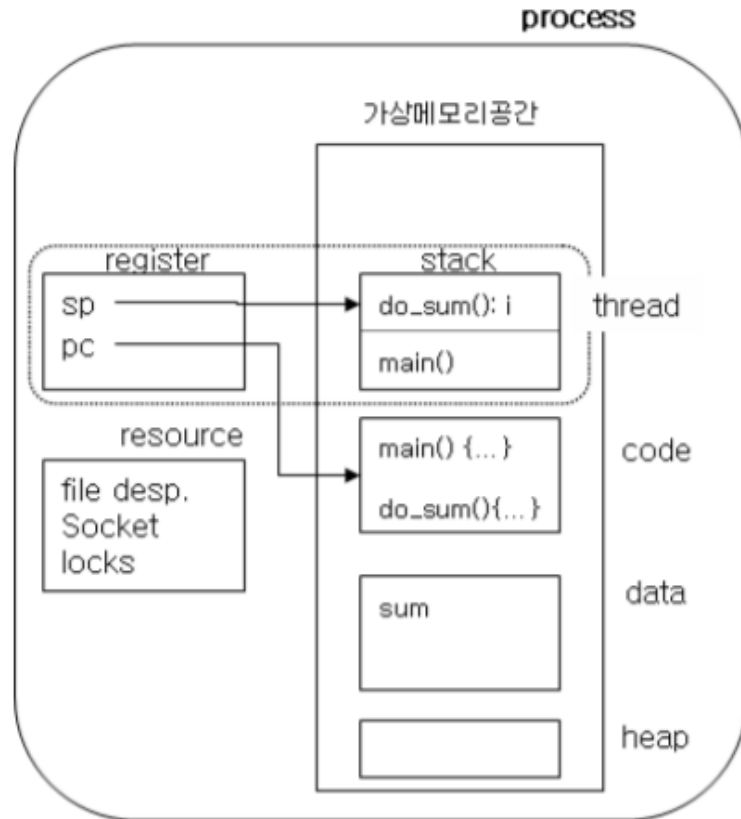
```
    printf("sum = %d\n", res);
```

```
    return 0;
```

```
}
```

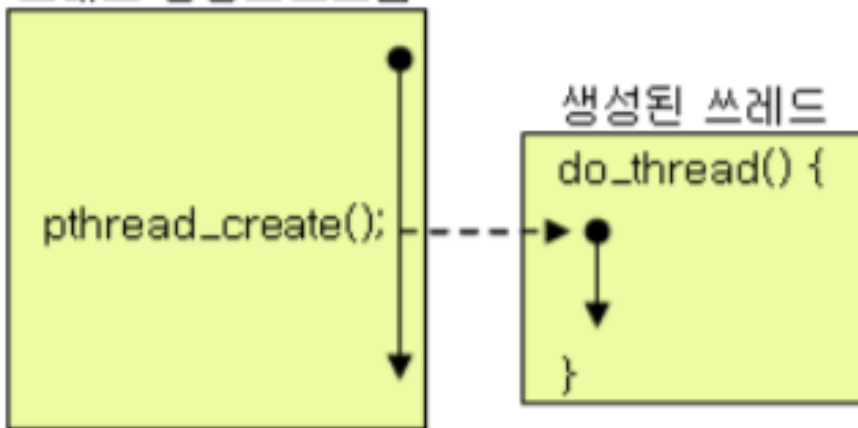


소켓 프로그래밍



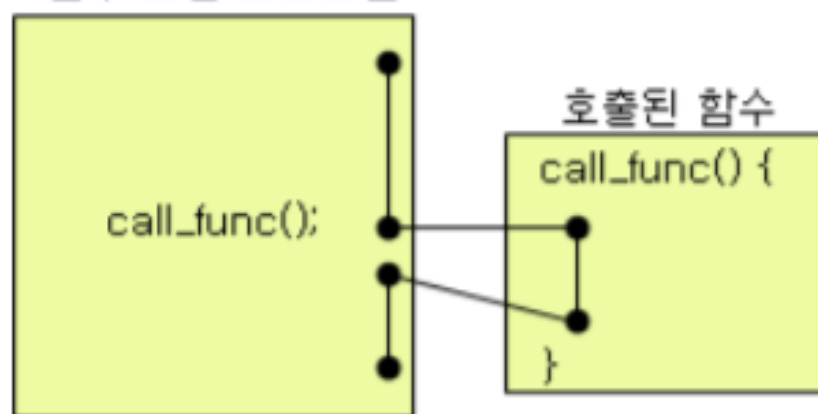
소켓 프로그래밍

쓰레드 생성 프로그램



<쓰레드 생성>

함수 호출 프로그램



<함수 호출>

소켓 프로그래밍

성질	fork로 생성한 프로세스	쓰레드	일반 함수
변수	모든 변수(지역변수, 전역변수) 공유 안됨	동일 프로세스의 쓰레드간 전역변수(global variable) 공유	전역변수(global variable) 공유
식별자	새로운 프로세스 식별자 부여	신규 쓰레드 식별자를 부여받지만 프로세스 식별자는 그대로 사용	동일 프로세스 식별자를 사용
통신	명시적(explicit) 통신방법(예: pipe)을 통해 통신	서로 공유하는 전역변수를 통해 통신 가능(공유변수 사용시 주의가 필요)	공유하는 공유변수를 통해 통신 가능(공유변수 사용시 별도의 주의가 필요하지 않음)
단일 CPU내 병렬수행	병렬수행	병렬수행	순차수행
다중 CPU내 병렬수행	동시 수행	동시 수행	순차수행

소켓 프로그래밍

```
//pthread_func.c
#include <stdio.h>
#include <pthread.h>

void* func_pthread(void*);

int
main(int argc, char **argv){

    int sts;
    pthread_t thread_id;
    void *t_return;

    if ( (sts = pthread_create(&thread_id, NULL, func_pthread, NULL)) != 0 ) {
        perror("pthread create error\n");
        exit(1);
    }

    printf("thread %x is created\n", thread_id);
    sleep(3);

    printf("main function exit\n");
    return 0;
}
```

```
void* func_pthread(void *arg)
{
    int i;

    while(1) {
        sleep(1);
        printf("thread executing....\n");
    }
}
```


소켓 프로그래밍

```
// pthread_join_func.c
#include <stdio.h>
#include <pthread.h>

void* do_sum(void *data)
{
    int i;
    int sum = 0;

    int max = *((int *)data);

    for (i = 0; i < max; i++)
    {
        sum = sum + i;
        printf("%d - Add %d\n", max, i);
        sleep(1);
    }
    printf("%d - sum(%d)\n", max, sum);
}
```

```
int main()
{
    int thr_id;
    pthread_t p_thread[3];

    int status;
    int a = 4;
    int b = 5;
    int c = 6;

    thr_id = pthread_create(&p_thread[0], NULL, do_sum, (void *)&a);
    thr_id = pthread_create(&p_thread[1], NULL, do_sum, (void *)&b);
    thr_id = pthread_create(&p_thread[2], NULL, do_sum, (void *)&c);

    pthread_join(p_thread[0], (void **) &status);
    pthread_join(p_thread[1], (void **) &status);
    pthread_join(p_thread[2], (void **) &status);

    printf("programing is end\n");
    return 0;
}
```

소켓 프로그래밍

```
//pthread_mutex.c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
```

```
int    ncount;
pthread_mutex_t  mutex = PTHREAD_MUTEX_INITIALIZER;
```

```
void* do_sum1(void *data)
```

```
{
    int    i;
    int    n = *((int *)data);

    for (i = 0; i < n; i++)
    {
        printf("sum1 : %d\n", ncount);

        pthread_mutex_lock(&mutex);
        ncount ++;
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
}
```

```
void* do_sum2(void *data)
```

```
{
    int i;
    int    n = *((int *)data);

    for (i = 0; i < n; i++)
    {
        printf("sum2 : %d\n", ncount);

        pthread_mutex_lock(&mutex);
        ncount ++;
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
}
```

```
int main()
```

```
{
    int    thr_id;
    pthread_t p_thread[2];
    int status;

    int    a ;

    ncount = 1;

    a = 9;
    thr_id = pthread_create(&p_thread[0], NULL, do_sum1, (void *)&a);
    sleep(1);

    a = 10;
    thr_id = pthread_create(&p_thread[1], NULL, do_sum2, (void *)&a);

    pthread_join(p_thread[0], (void **) &status);
    pthread_join(p_thread[1], (void **) &status);

    status = pthread_mutex_destroy(&mutex);
    printf("programing is end");
    return 0;
}
```

소켓 프로그래밍

```
// talk_server_pthread
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <signal.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <pthread.h>

void* do_keyboard(void *);
void* do_socket(void *);
char quit[] = "exit";
pthread_t pid[2];

int main(int argc, char *argv[]) {

    int    thr_id;
    int    status;
    int listenSock, connSock;
    struct sockaddr_in client_addr, server_addr;

    int len;
    if (argc < 2) {
        printf("Usage: talk_server port_num ");
        return -1;
    }
    if((listenSock=socket(PF_INET, SOCK_STREAM, 0)) < 0) {
        printf("Server: Can't open socket\n");
        return -1;
    }
}
```

```
bzero((char *)&server_addr, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
server_addr.sin_port = htons(atoi(argv[1]));

if(bind(listenSock, (struct sockaddr *)&server_addr, sizeof(server_addr))<0){
    printf("talk Server Can't bind %d\n");
    return -1;
}
listen(listenSock, 1);
len = sizeof(client_addr);
if((connSock = accept(listenSock,(struct sockaddr *)&client_addr, &len))<0){
    printf("Talk Server failed in accepting\n");
    return -1;
}
printf("Talk Server accept new request\n");
thr_id = pthread_create(&pid[0], NULL, do_keyboard, (void *)connSock);
thr_id = pthread_create(&pid[1], NULL, do_socket, (void *)connSock);

pthread_join(pid[0], (void **) &status);
pthread_join(pid[1], (void **) &status);

close(listenSock);
close(connSock);
}
```

소켓 프로그래밍

```
void* do_keyboard(void *data)
{
    int    n;
    char    sbuf[BUFSIZ];
    int    connSock = (int) data;

    while( (n = read(0, sbuf, BUFSIZ)) > 0) {
        if(write(connSock, sbuf, n) != n) {
            printf("Talk Server fail in sending\n");
        }
        if(strncmp(sbuf, quit, 4) == 0) {
            pthread_kill(pid[1], SIGINT);
            break;
        }
    }
}
```

```
void* do_socket(void *data)
{
    int n;
    char    rbuf[BUFSIZ];
    int    connSock = (int) data;

    while(1) {
        if((n = read(connSock, rbuf, BUFSIZ)) > 0) {
            rbuf[n] = '\0';
            printf("%s", rbuf);
            if (strncmp(rbuf, quit, 4) == 0) {
                pthread_kill(pid[0], SIGINT);
                break;
            }
        }
    }
}
```

소켓 프로그래밍

11주 과제

1. `ps -lL` 를 사용하여 thread가 사용되는 화면을 캡처하고 process 와 thread 의 차이를 설명하시오. (`echo_server_fork.c` 와 `echo_server_pthread.c`)
2. Thread를 사용할때 `join` 을 해야하는 이유를 설명하시오. (잘못된 예를 들어 설명하시오.)
3. `mutex` 는 어떤 경우에 사용해야 하는지 설명하시오.
4. `thread_test.c` 에는 함수 2개를 이용하여 thread 를 실행하는데 이것을 하나의 함수로 실행할 수 있도록 변경하시오.